

Relazione Algoritmi e strutture dati

Progetto 2023/2024

Esercizio 1 - Merge Sort e Quick Sort

Le misure sono state effettuate sul dataset records.csv: una prova per configurazione sulla stessa macchina. I tempi CPU, rilevati con clock(), riguardano soltanto l'ordinamento ed escludono la lettura e la scrittura del CSV.

10 milioni di records:

	QuickSort	MergeSort
Interi	12,564 s	3,629 s
Stringhe	6,131 s	4,952 s
Double	11,353 s	3,946 s

La tabella riporta i tempi di QuickSort e MergeSort sui primi 10 milioni di record del file. QuickSort sceglie la mediana di tre come pivot, partiziona i valori in tre gruppi e applica la ricorsione solo alla partizione più piccola. MergeSort divide ricorsivamente i record e fonde le partizioni ordinate usando un buffer ausiliario.

Sui 10 milioni di record MergeSort è più rapido: QuickSort impiega circa 1,24 volte il tempo sulle stringhe, 3,46 sugli interi e 2,88 sui double. Le prestazioni dipendono quindi anche dalla chiave.

20 milioni di records:

	QuickSort	MergeSort
Interi	28,491 s	8,225 s
Stringhe	12,375 s	10,082 s
Double	22,619 s	8,450 s

Raddoppiando i record, i tempi misurati crescono di circa 2,0–2,3 volte per entrambi gli algoritmi: non rimangono costanti. Il rapporto varia con la chiave scelta. Tutte e sei le esecuzioni sul dataset completo da 20 milioni di record sono terminate senza errori.

Esercizio 2 - Edit distance

Dynamic distance:

La funzione ricorsiva `edit_distance_dyn` usa una tabella di memoizzazione dimensionata alle due stringhe: evita di ricalcolare gli stessi sottoproblemi e restituisce la distanza minima per sole inserzioni e cancellazioni.

Il programma usa l'intero dizionario: riconosce le parole presenti e, per quelle assenti, mostra tutte le voci alla minima distanza positiva. La ricerca di una parola esatta usa il dizionario ordinato.

Estratto dell'output del programma su `dictionary.txt` e `correctme.txt`:

```
$ ./bin/main_ex2 dictionary.txt correctme.txt
La parola 'quando' è corretta.
La parola 'avevo' è corretta.
Parola: 'cinque'
|Edit distance minima|: 2
Parole possibili:
- cinque
- cive
...
Parola: 'squola'
|Edit distance minima|: 1
Parole possibili:
- suola
```

Distance:

I test automatici confrontano le funzioni ricorsiva e dinamica su stringhe brevi e sui casi limite. Le due funzioni devono produrre la stessa distanza.

Estratti dell'esecuzione dei test Unity, inclusi il confronto tra le due funzioni e i controlli sui file di input:

```
$ ./bin/test_ex2
test_edit_distance_empty_strings:PASS
test_edit_distance_same_strings:PASS
test_edit_distance_one_deletion:PASS
test_edit_distance_one_insertion:PASS
test_edit_distance_recursive_vs_dynamic:PASS
test_edit_distance_dynamic_over_hundred_characters:PASS
```

```
test_dictionary_loads_more_than_ten_thousand_words:PASS
test_text_loader_does_not_truncate_large_files:PASS

12 Tests 0 Failures 0 Ignored
OK
```

Tutti i 12 test automatici dell'esercizio 2 terminano senza errori.

Esercizio 3 - PriorityQueue

La coda di priorità utilizza due strutture:

- **ArrayList**: contiene gli elementi della coda.
- **HashMap**: contiene come chiavi gli elementi della coda, che hanno come valore il proprio indice nell'arrayList (heap), in modo da poter accedere ai vari elementi rispettando le complessità.

Nella seguente tabella sono riportate le complessità richieste e le complessità raggiunte durante l'implementazione delle funzioni.

Tabella complessità funzioni

	Richiesta	Implementazione
empty()	$O(1)$	$O(1)$
push()	$O(\log N)$	$O(\log N)$
contains()	$O(1)$	$O(1)$
top()	$O(1)$	$O(1)$
pop()	$O(\log N)$	$O(\log N)$
remove()	$O(\log N)$	$O(\log N)$

La coda è stata implementata per elementi generici, in cui in base al comparatore passato è possibile definire il comportamento dell'heap (per classi di oggetti specifici è anche necessario definire un comparatore specifico di quella classe) :

Max-heap: viene definita una coda in cui gli si passa un comparatore della classe standard di Java **reverseOrder**, in cui nell'heap il primo elemento è quello con maggiore priorità.

Min-heap: viene definita una coda in cui gli si passa un comparatore della classe standard di Java ***naturalOrder***, in cui nell'heap il primo elemento è quello con minore priorità.

E' presente anche un **menù interattivo** in cui è possibile eseguire le operazioni sulla coda di priorità, di seguito un esempio del funzionamento:

Esempio

```
Selezionare l'esercizio:
0) Esercizio 3
1) Esercizio 4
Scelta: 0
Esercizio 3 - Code di priorità
Determinare il tipo di coda da testare:
0) Max-heap
1) Min-heap
Scelta: 0
Seleziona il tipo della coda da utilizzare:
0) Intero
1) Double
2) String
Scelta: 0
Inserisci i valori per la coda (separati dalla virgola):
9,7,10,3,2,16,14,4,8,1
Coda creata!

Operazioni sulla coda:
0) Controlla se è vuota
1) Inserisci elementi
2) Contiene gli elementi (Inserisci gli elementi da cercare).
3) Primo elemento della coda.
4) Rimuovi il primo elemento della coda.
5) Rimuovi gli elementi dalla coda (Inserisci gli elementi da eliminare).
6) Stampa l'heap.
7) Stampa l'hashmap.
8) Termina l'esecuzione
Scelta: 6
Contenuto della coda:
16
8
14
7
2
9
10
3
4
1
```

In questo esempio è stata creata una coda max-heap con i valori 9, 7, 10, 3, 2, 16, 14, 4, 8 e 1.

L'heap risultante è 16, 8, 14, 7, 2, 9, 10, 3, 4, 1.

Per ogni nodo, il genitore ha priorità almeno pari a quella dei figli: la proprietà dell'heap è quindi rispettata. Gli stessi valori possono produrre più configurazioni valide.

Esercizio 4 - Grafi sparsi e foreste ricoprenti minime

La struttura principale del grafo è una mappa delle adiacenze: costituisce l'unica fonte di verità per nodi e archi. Un contatore separato conserva soltanto il numero di archi.

adjacencyMap: a ogni nodo associa una mappa dai vicini agli oggetti arco. Nei grafi non diretti lo stesso arco compare in entrambe le adiacenze, senza creare due oggetti distinti.

Graficamente:

$A \rightarrow B \rightarrow \text{ARCO}(A,B)$

Key Key Value

| Mappa annidata |

Value

Tabella complessità funzioni

	Richiesta	Implementazione
isDirected()	O(1)	O(1)
isLabelled()	O(1)	O(1)
addNode(V a)	O(1)	O(1)
addEdge(V a, V b, L l)	O(1)	O(1)
containsNode(V a)	O(1)	O(1)
containsEdge(V a, V b)	O(1)	O(1)
removeNode(V a)	O(N)	O(N)
removeEdge(V a, V b)	O(1)	O(1)
numNodes()	O(1)	O(1)
numEdges()	O(N)	O(1)
getNodes()	O(N)	O(1)
getEdges()	O(N)	O(N + E)

getNeighbours(V a)	O(1)	O(1)
getLabel(V a, V b)	O(1)	O(1)

getEdges(): costruisce una collezione non modificabile degli archi a partire dalle adiacenze. La visita costa $O(N + E)$, dove N è il numero di nodi ed E quello degli archi. La collezione restituita è uno snapshot costruito dalle adiacenze. Il grafo mantiene una sola rappresentazione degli archi, evitando duplicazioni di stato.

numEdges(): restituisce in $O(1)$ il contatore aggiornato soltanto quando un arco viene aggiunto o rimosso con successo. Il contatore non richiede la scansione delle adiacenze e conta una sola volta gli archi non diretti. Nei grafi non diretti le due direzioni fanno riferimento allo stesso arco; nei grafi diretti viene memorizzata soltanto la direzione richiesta.

getNodes(): restituisce in $O(1)$ una vista non modificabile delle chiavi della mappa; iterare tutti i nodi richiede $O(N)$. Non è mantenuta una lista separata dei nodi: aggiunte e rimozioni sono rappresentate direttamente dalla mappa delle adiacenze.

Prim

Sul grafo italiano l'algoritmo di Prim produce una foresta minima con 18.640 nodi e 18.637 archi, dal peso totale di 89.939,913 km. Gli archi sono emessi come CSV e il riepilogo sullo standard error.

```
$ java -cp ex3-4/bin Prim.Prim italian_dist_graph.csv
casale di pari,il belagaio,6562.496730052492
nebbiu,venas,3857.095606855183
giano dell'umbria,villa fabbri,2298.1001393987626
monsagrati,valpromaro,4413.486500572089
carbona,pieve fosciana,3706.460489842959
laureana di borrello,plaesano,1853.2858423846617
magliano,regnano,3238.66952590965
castello di raganzuolo,manzana,2010.9337117619796
...
Nodes: 18640, edges: 18637, weight: 89939.913 km
```